

CIS241

System-Level Programming and Utilities

Pattern scanning and processing language (`awk`)

Erik Fredericks, frederer@gvsu.edu

Fall 2025

Based on material provided by
Erin Carrier, Austin Ferguson,
and Katherine Bowers

```
\ ————— BACKSLASH
\\ ————— REAL BACKSLASH
\\\ ————— REAL REAL BACKSLASH
\\\\ ————— ACTUAL BACKSLASH, FOR REAL THIS TIME
\\\\\\ ————— ELDER BACKSLASH
\\\\\\\\ ————— BACKSLASH WHICH ESCAPES THE SCREEN AND ENTERS YOUR BRAIN
\\\\\\\\\\ ————— BACKSLASH SO REAL IT TRANSCENDS TIME AND SPACE
\\\\\\\\\\\\ ————— BACKSLASH TO END ALL OTHER TEXT
\\\\\\\\\\\\\\... — THE TRUE NAME OF BA'AL, THE SOUL-EATER
```

awk

Pattern scanning and processing language

- Data driven, not procedural
 - Describe data to work with
 - Tell it what to do when pattern found!

Good for data in columns

- `gawk` is the GNU implementation - what we'll use

gawk program

```
BEGIN { ... initialization gawk statements ... }  
gawk comamnds to run on each line of the file  
END { ... finalization gawk statements ... }
```

Each command has the form: `pattern { action }`

- `action` = one or more statements enclosed in braces
- Pattern can be regex
- No pattern - action performed on all lines

Fields

Each line is made up of fields

- **Field separator distinguished fields**
 - Default - `space`
 - Change value of `FS` to use another
 - or use `-F` to change
- **Reference field by `$#` (`$0` is entire line)**
- **`NF` = number of fields on current line**
- **`NR` = record number of current line**

Sample data

chevy	impala	1985	85	1550
ford	explorer	2003	25	9500

- Record is a row, field is a cell!

Running `gawk`

```
gawk [options] program [input_files]
```

```
gawk -f program_file [input_files]
```

Can also create a script telling it to run with `gawk` instead!

- `#!/usr/bin/env gawk -f`
 - Also can use path directly to `gawk`
 - Script needs to be executable!

Patterns

Pattern can be regex (`/regex/`)

- `~` used for matching regex
- `!~` tests for not matching regex

Pattern can also compare field or variable to value:

- `==`, `!=`, `<`, `<=`, `>`, `>=`

`BEGIN` and `END` are special patterns

Nothing for pattern -> applies to all records

Can combine patterns with `&&` (and), `||` (or)

Variables

Can hold strings/numeric values

Typically initialized in `BEGIN`

Default -> initialized to empty string or `0`

Standard arithmetic operators available

<https://medium.com/@redswitches/the-gawk-command-in-linux-with-10-examples-092900b06ca5>

```
gawk '/[e,E]$/ {print}' words.txt
```